

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 21
Graphs (II)

图论

Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Outline

- Topological Sorting
  - Breadth First
  - Depth First
- Euler Paths and Circuits*

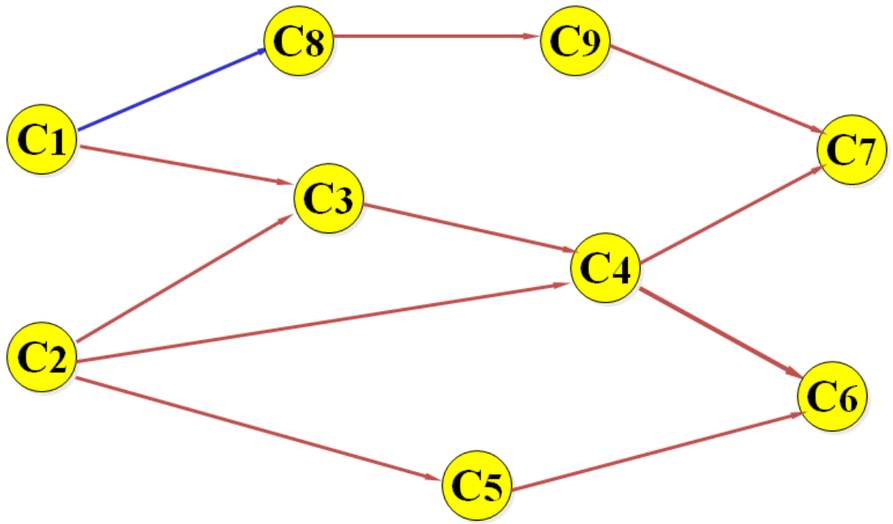
Topological Sorting (拓扑排序)

- Let *G* be a directed graph with no cycles (DAG). A topological order for *G* is a sequential listing of all the vertices in *G* such that, for all vertices $v, w \in G$, if there is an edge from *v* to *w*, then *v* precedes *w* in the sequential listing.
- The canonical application of topological sorting (topological order) is in scheduling a sequence of jobs or tasks; topological sorting algorithms were first studied in the early 1960s in the context of the PERT technique for scheduling in project management.

A DAG Example

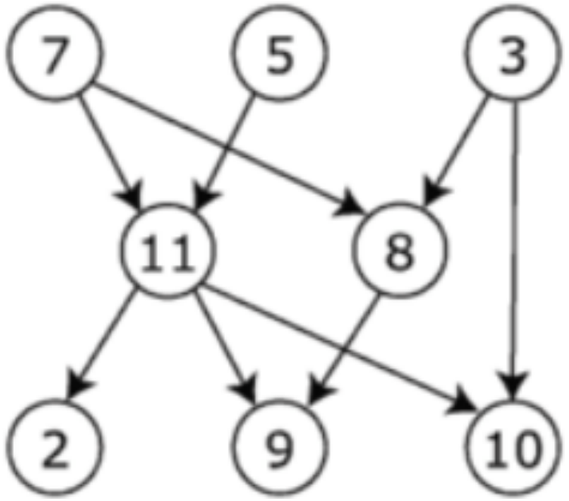
课程代号	课程名称	先修课程
C_1	高等数学	
C_2	程序设计基础	
C_3	离散数学	C_1, C_2
C_4	数据结构	C_3, C_2
C_5	高级语言程序设计	C_2
C_6	编译方法	C_5, C_4
C_7	操作系统	C_4, C_9
C_8	普通物理	C_1
C_9	计算机原理	C_8

学生课程学习工程图



Example of Topological Sorting

- The graph shown in the right has many valid topological sorts, including:
  - 7, 5, 3, 11, 8, 2, 9, 10 (visual left-to-right, top-to-bottom)
  - 3, 5, 7, 8, 11, 2, 9, 10 (smallest-numbered available vertex first)
  - 3, 7, 8, 5, 11, 10, 2, 9
  - 5, 7, 3, 8, 11, 10, 9, 2 (least number of edges first)
  - 7, 5, 11, 3, 10, 8, 9, 2 (largest-numbered available vertex first)
  - 7, 5, 11, 2, 3, 8, 9, 10

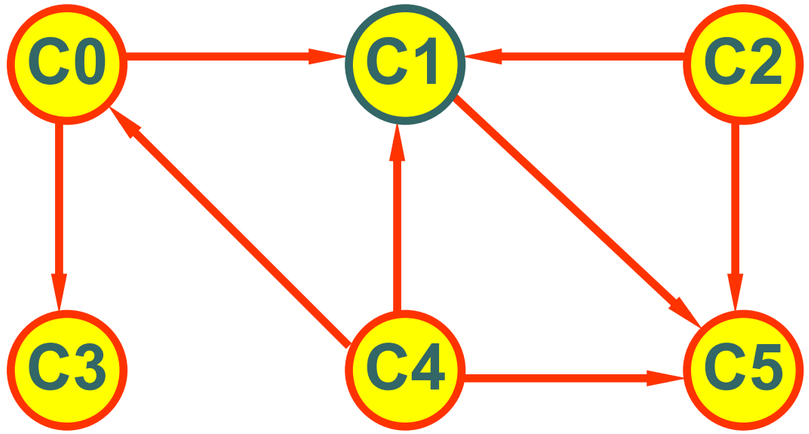


Topological Sorting: Breadth-First Algorithm

- First, find a complete set of “source vertices" which have no incoming edges; at least one such node must exist if the graph is acyclic. Deleting all the outgoing edges of these source vertices will create new source vertices, which can then sit comfortably to the immediate right of the first set. We repeat until all vertices are accounted for.

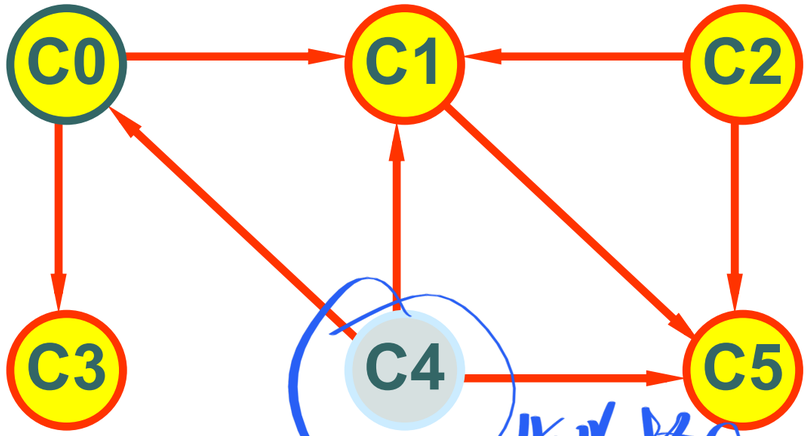
```
L ← Empty list that will contain the sorted elements
S ← Set of all nodes with no incoming edges
while S is non-empty do
    remove a node n from S
    add n to tail of L
    for each node m with an edge e from n to m do
        remove edge e from the graph
        if m has no other incoming edges then
            insert m into S
if graph has edges then
    return error (graph has at least one cycle)
else
    return L (a topologically sorted order)
```

Breadth-First Example for Topological Sorting



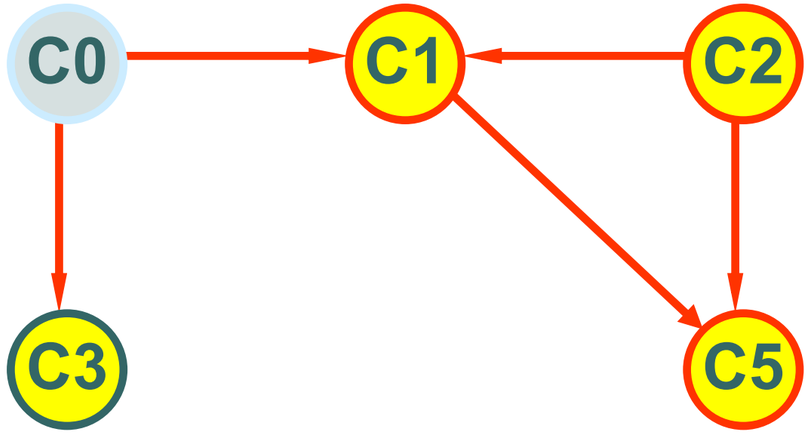
(a) DAG

无环图

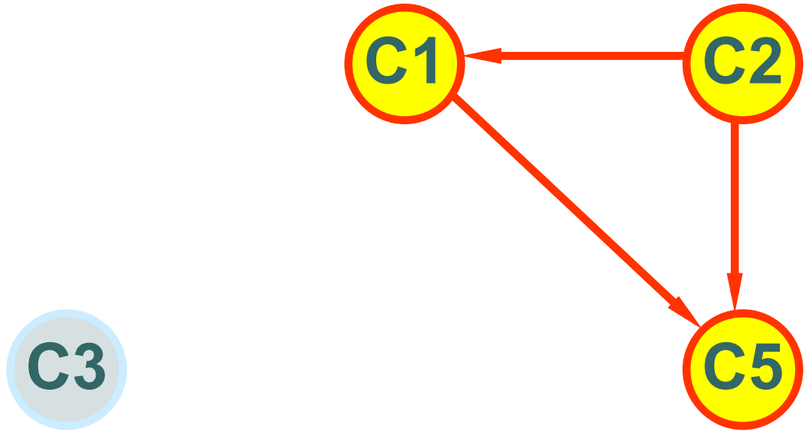


(b) Output C4

①先选择C4 ②删除C4及所有出发边

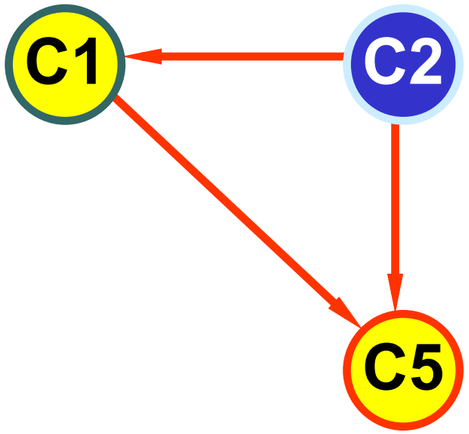


(c) Output C0

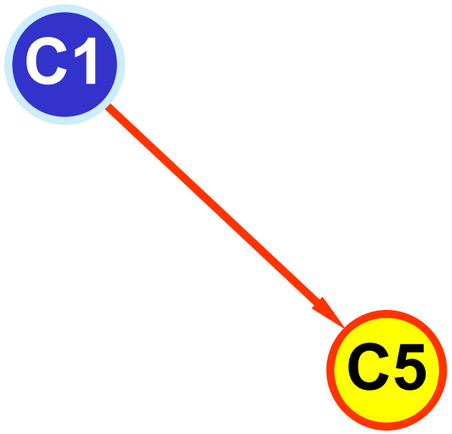


(d) Output C3

Breadth-First Example for Topological Sorting



(e)Output C2



(f)Output C1



(g)Output C5

(h) Finish

- The topological order: C_4 , C_0 , C_3 , C_2 , C_1 , C_5

Topological Sorting: Breadth-First Algorithm

- If the graph was a DAG, a solution is contained in the list L (the solution is not unique). Otherwise, the graph has at least one cycle and therefore a topological sorting is impossible.
- The usual algorithms for topological sorting have running time linear in the number of nodes plus the number of edges, asymptotically, O(|V|+|E|).

Topological Sorting: Breadth-First Algorithm

- 有向图的存储形式的不同，拓扑排序算法的实现也不同。
- 基于邻接矩阵表示的存储结构，设A为有向图G的邻接矩阵，则有：
  - 找图G中无前驱的顶点——在A中找到值全为0的列
  - 删除以i为起点的所有弧——将矩阵中i对应的行全部置为0。
- 基于邻接表的存储结构：入度为零的顶点即为没有前驱的顶点，我们可以附设一个存放各顶点入度的数组indegree[]，则有：
  - 找G中无前驱的顶点——查找indegree[i]为零的顶点vi；
  - 删除以i为起点的所有弧——对链在顶点i后面的所有邻接顶点k，将对应的indegree[k]减1。

Topological Sorting: Breadth-First Algorithm

```
template <int graph_size>
void Digraph<graph_size> :: breadth_sort(List<Vertex> &topological_order)
/* Post: The vertices of the Digraph are arranged into the List topological_order
         which is found with a breadth-first traversal of those vertices that do not
         belong to a cycle.
   Uses: Methods of classes Queue and List. */
{
   topological_order.clear();
   Vertex v, w;
   int predecessor_count[graph_size];
   for (v = 0; v < count; v++) predecessor_count[v] = 0;
   for (v = 0; v < count; v++)
      for (int i = 0; i < neighbors[v].size(); i++) {
                                   //    Loop over all edges v — w.
         neighbors[v].retrieve(i, w);
         predecessor_count[w] ++;
      }
   Queue ready_to_process;
   for (v = 0; v < count; v++)
      if (predecessor_count[v] == 0)
         ready_to_process.append(v);
   while (!ready_to_process.empty()) {
      ready_to_process.retrieve(v);
      topological_order.insert(topological_order.size(), v);
      for (int j = 0; j < neighbors[v].size(); j++) {  //    Traverse successors of v.
         neighbors[v].retrieve(j, w);
         predecessor_count[w] --;
         if (predecessor_count[w] == 0)
            ready_to_process.append(w);
      }
      ready_to_process.serve();
   }
}
```

Topological Sorting: Depth-First Algorithm

Topological-Sort (G)

1. Call DFS(G) to compute finishing times $f[v]$ for all $v \in V$
2. As each vertex is finished, insert it onto the front of a linked list
3. **return** the linked list of vertices

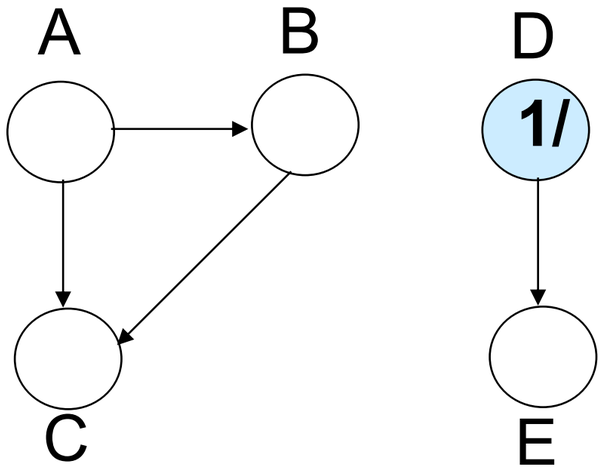
i.e., sorted from largest to smallest by finishing time

```
template <int graph_size> void Digraph<graph_size> ::
    depth_sort(List<Vertex> &topological_order)
/* Post: The vertices of the Digraph are placed into List topological_order with a
         depth-first traversal of those vertices that do not belong to a cycle.
   Uses: Methods of class List, and function recursive_depth_sort to perform depth-
         first traversal. */
{ bool visited[graph_size];
  Vertex v;
  for (v = 0; v < count; v++) visited[v] = false;
  topological_order.clear( );
  for (v = 0; v < count; v++)
    if (!visited[v])              // Add v and its successors into topological order.
      recursive_depth_sort(v, visited, topological_order);
}
```

Topological Sorting: Depth-First Algorithm

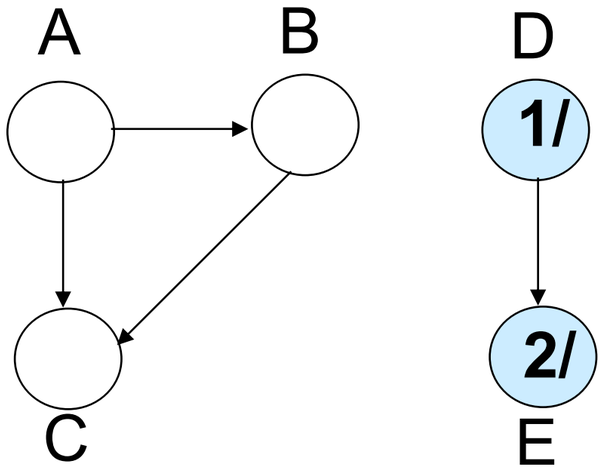
```
template <int graph_size> void Digraph<graph_size> ::
recursive_depth_sort(Vertex v, bool *visited, List<Vertex> &topological_order)
/* Pre:   Vertex v of the Digraph does not belong to the partially completed List topo-
          logical_order.
   Post:  All the successors of v and finally v itself are added to topological_order
          with a depth-first search.
   Uses:  Methods of class List and the function recursive_depth_sort.  */
{  visited[v] = true;
   int degree = neighbors[v].size( );
   for (int i = 0;  i < degree;  i++) {
      Vertex w;                          //   A (neighboring) successor of v
      neighbors[v].retrieve(i, w);
      if (!visited[w])                   //   Order the successors of w.
         recursive_depth_sort(w, visited, topological_order);
   }
   topological_order.insert(0, v);   //   Put v into topological_order.
}
```

Topological Sorting: Depth-First Algorithm



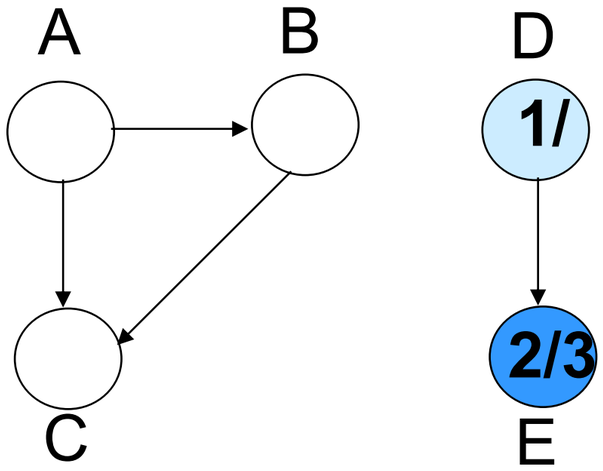
Linked List:

Topological Sorting: Depth-First Algorithm



Linked List:

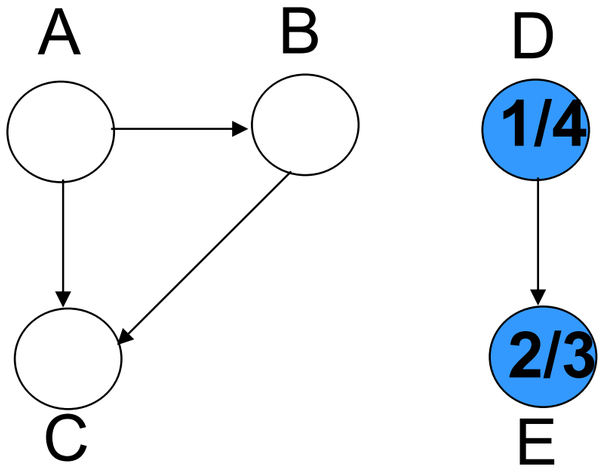
Topological Sorting: Depth-First Algorithm



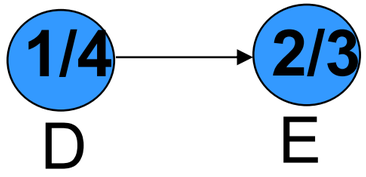
Linked List:



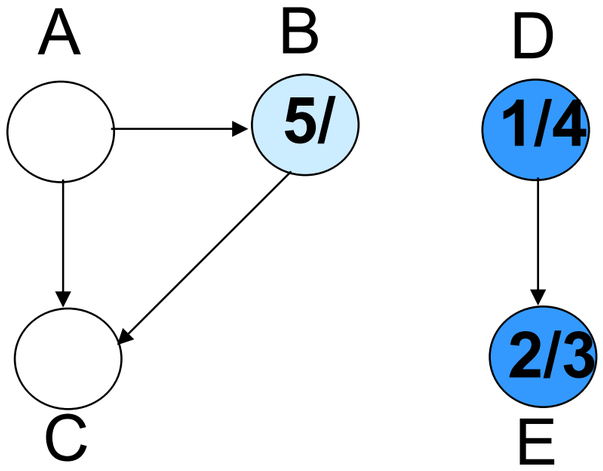
Topological Sorting: Depth-First Algorithm



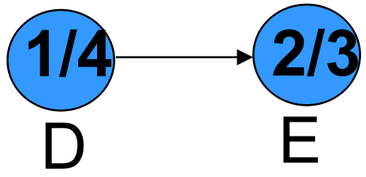
Linked List:



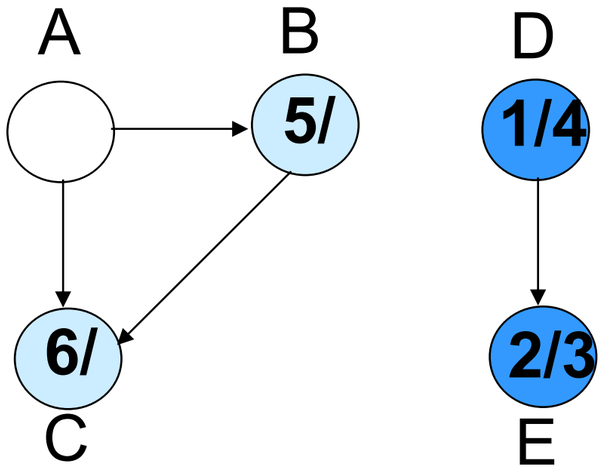
Topological Sorting: Depth-First Algorithm



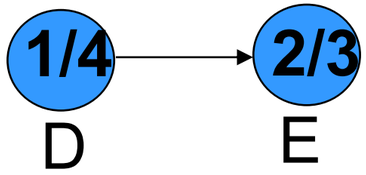
Linked List:



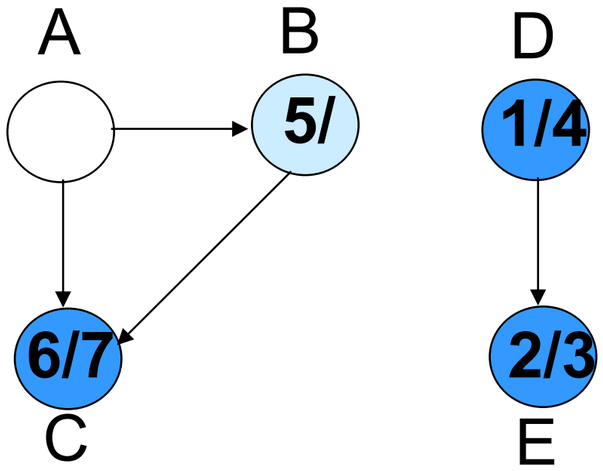
Topological Sorting: Depth-First Algorithm



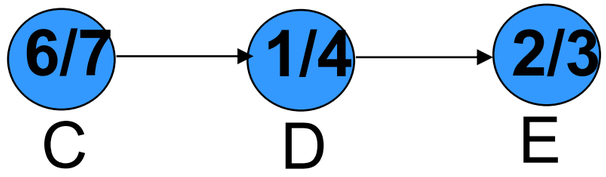
Linked List:



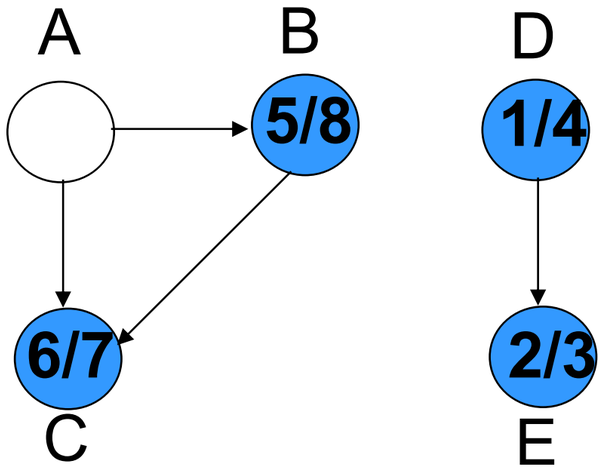
Topological Sorting: Depth-First Algorithm



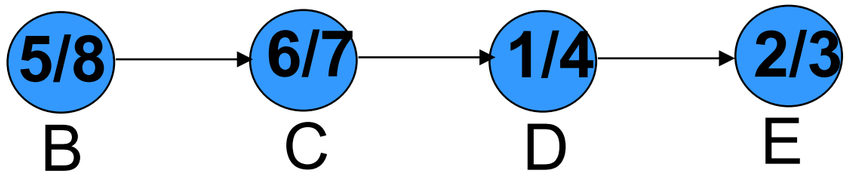
Linked List:



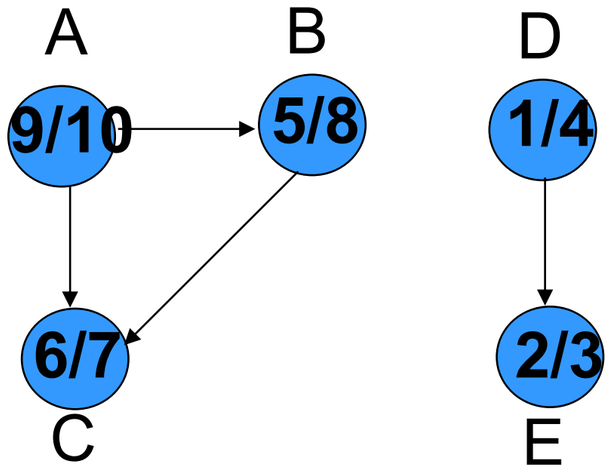
Topological Sorting: Depth-First Algorithm



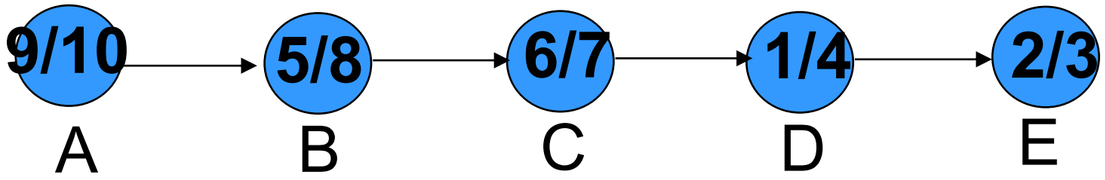
Linked List:



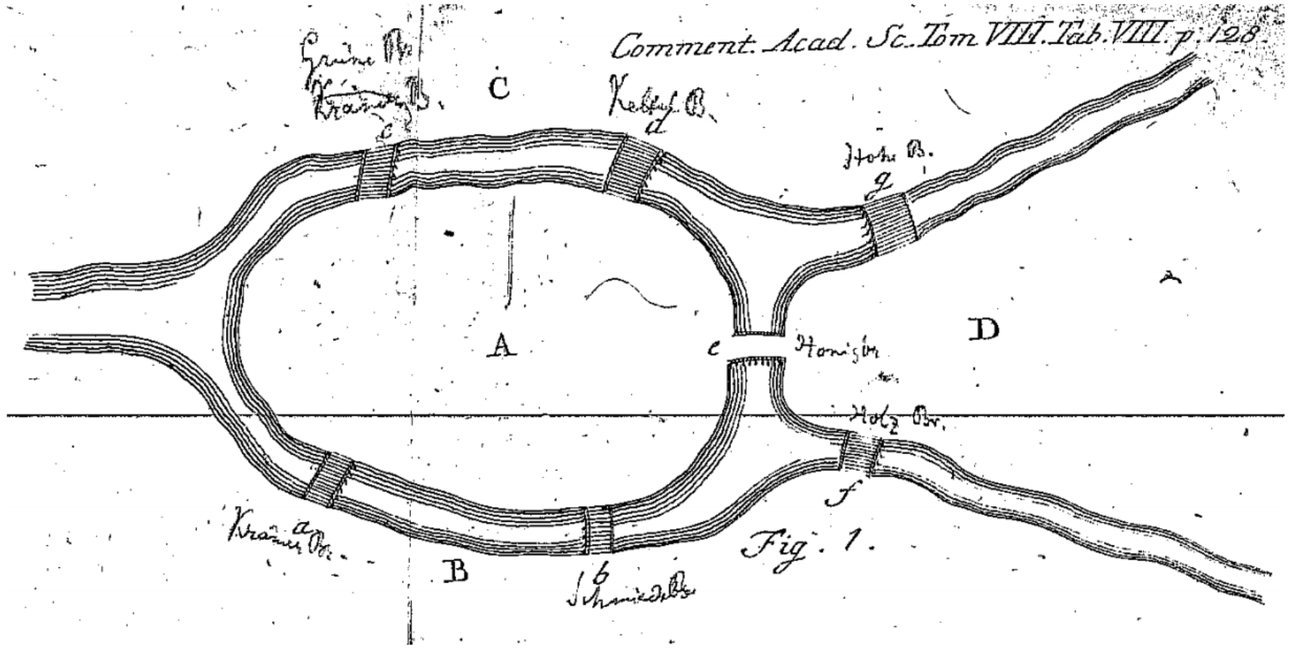
Topological Sorting: Depth-First Algorithm



Linked List:



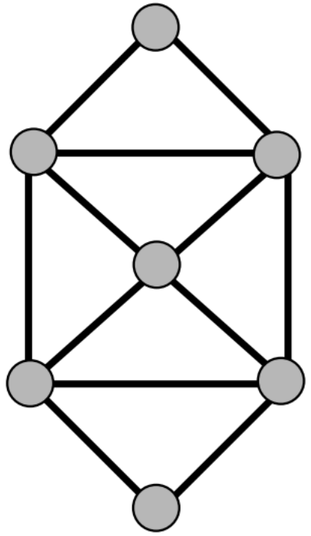
The Seven Bridges of Königsberg

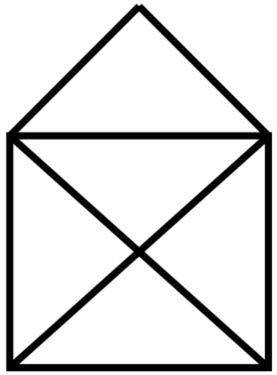


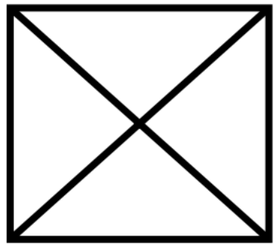
- An illustration from Euler's 1741 paper on the subject.

Puzzles

- Can you traverse all edges exactly once, starting and finishing at the same vertex?







Euler Paths and Circuits

- Euler Path: a path through a graph that visits each edge exactly once.
- Euler Circuit: an Euler tour that starts and ends at the same vertex.
- Observations:
  - An Euler circuit is only possible if the graph is connected and each vertex has even degree (# of edges onto vertex).
  - Why?
  - At every vertex, need one edge to get in and one edge to get out!

The Handshaking Theorem

- Theorem: In every graph, the sum of the degrees of all vertices equals twice the number of edges.
- If there are n vertices $V_1, \ldots, V_n$, with degrees $d_1, \ldots, d_n$, and there are e edges, then

$$d_1 + d_2 + \cdots + d_{n-1} + d_n = 2e$$

- Therefore, $d_1 + d_2 + \cdots + d_n$ must be an even number.
- The numbers $d_1, d_2, \cdots, d_n$ must include an even number of odd numbers.
- Every graph has an even number of odd vertices.

Euler Circuit Theorem

- If a graph is *connected* and *every vertex is even*, then it has an Euler circuit (at least one, usually more).
- If a graph has *any odd vertices*, then it does not have an Euler circuit.

Euler Path Theorem

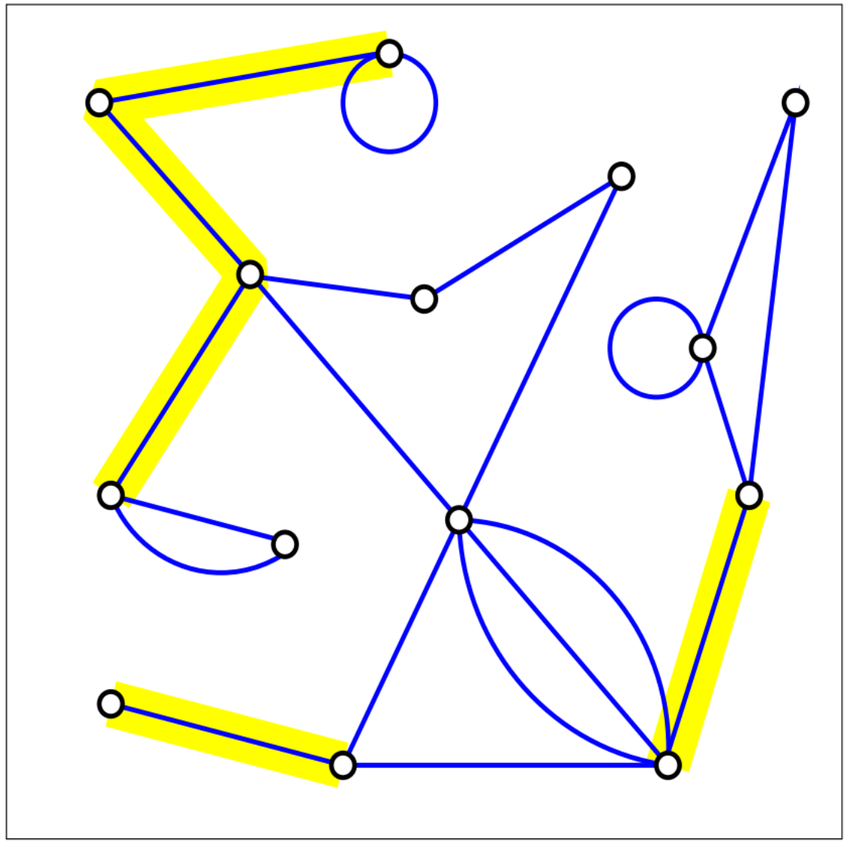
- If a graph is *connected* and has exactly two odd vertices, then it has an Euler path (at least one, usually more). Any such path must start at one of the odd vertices and end at the other one.
- If a graph has more than two odd vertices, then it cannot have an Euler path.

Finding Euler Circuits

- Given a graph G = (V,E), find an Euler circuit in G.
- Can check if one exists in O(|V|) time (check degrees). 从理论判断：度数
- To find an Euler path or an Euler circuit
  - Fleury's Algorithm
  - DFS Algorithm

Fleury's Algorithm: Bridges

- Removing a single edge from a connected graph can make it disconnected. Such an edge is called a bridge.



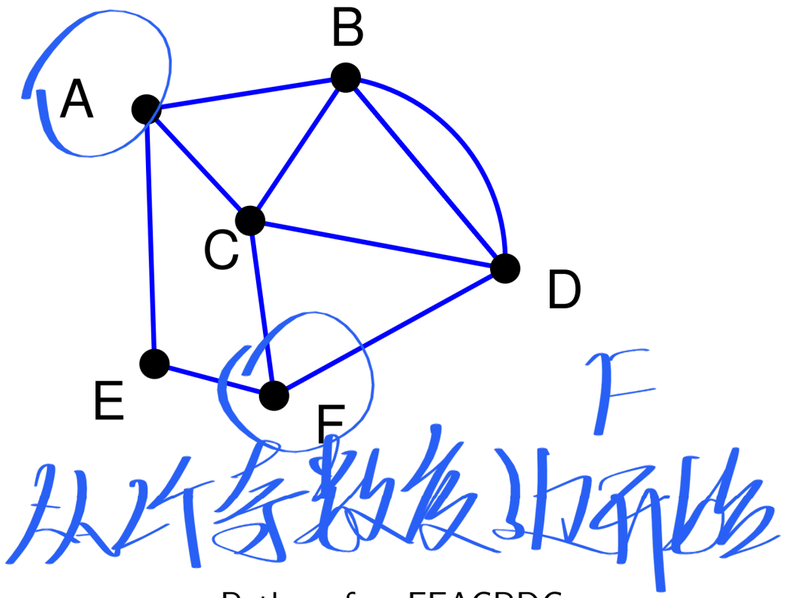
Fleury's Algorithm

- Idea: Don't burn your bridges.

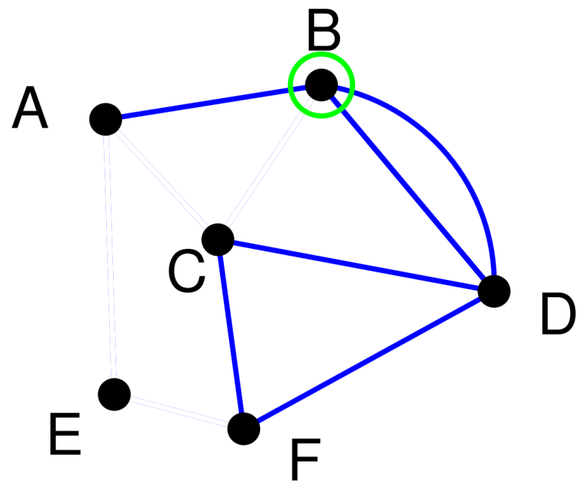
0或2个奇数度的点：判断是否连通

1. Make sure the graph has either 0 or 2 odd vertices.
2. If there are 0 odd vertices, start anywhere. If there are 2 odd vertices, start at one of them.
3. Follow edges one at a time. If you have a choice between a bridge and a non-bridge, always choose the non-bridge.
4. Stop when you run out of edges.

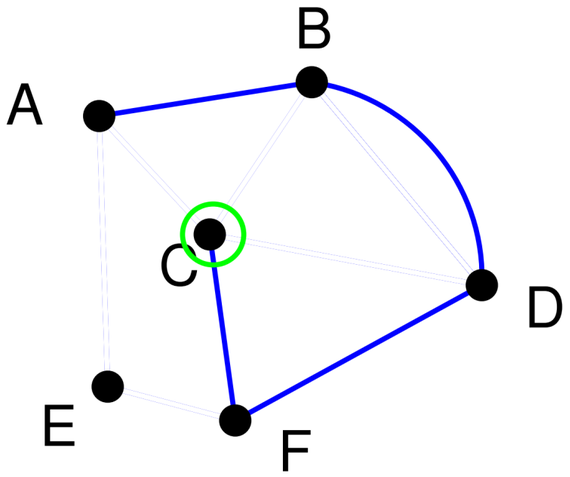
Example



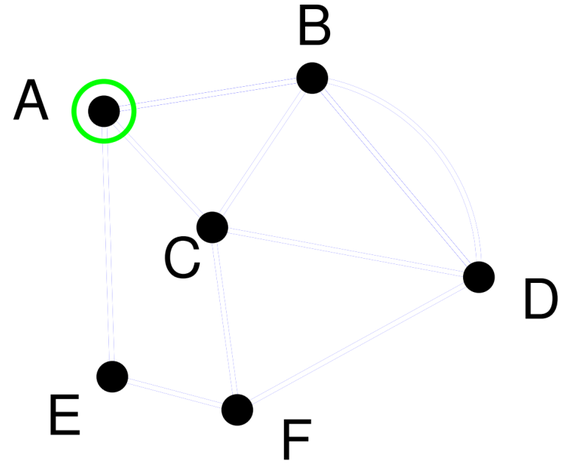
Path so far: FEACB



Path so far: FEACBDC



Euler Path: FEACBDCFDBA



Hierholzer's Algorithm

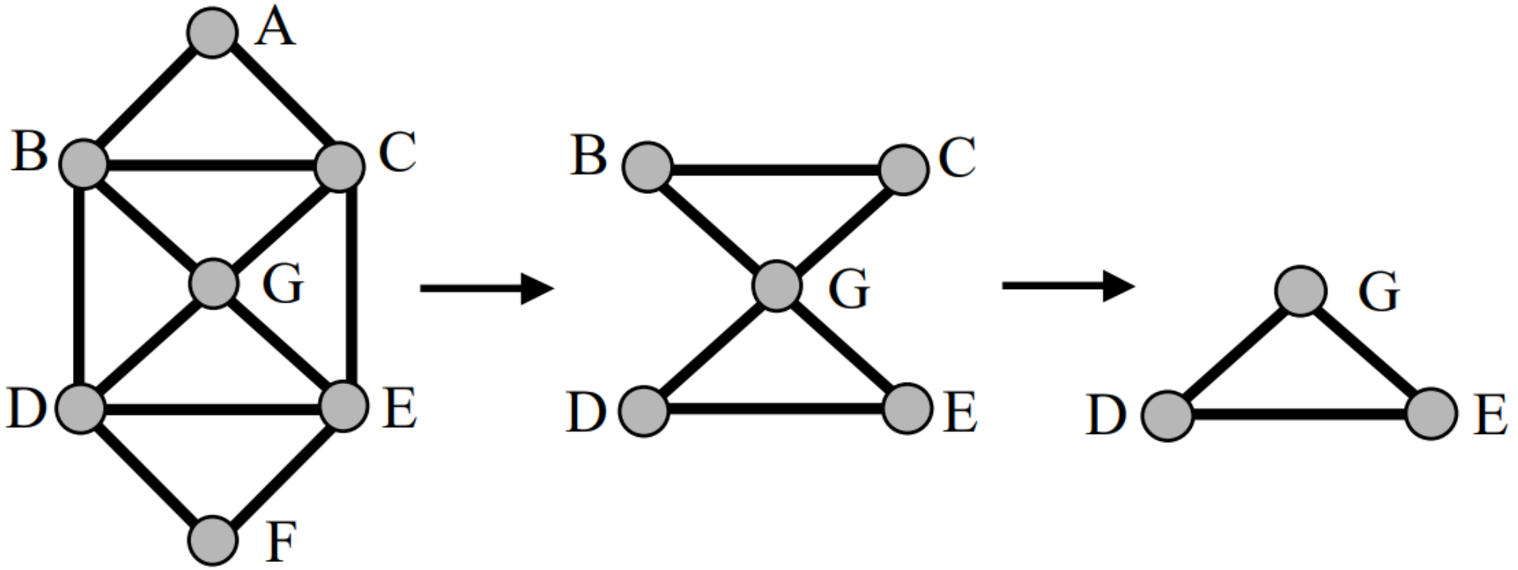
- DFS and then Splice

1. Perform a depth-first search (DFS) from a vertex until you are back at this vertex.
2. Pick a vertex on this path with an unused edge and repeat 1.
3. Splice all these paths into an Euler circuit.



- Running time = O(|V|+|E|).

Example



DFS(A) :
A B D F E C A

DFS(B) :
B G C B

Splice at G

DFS(G) :
G D E G

Splice at B → A B G C B D F E C A

A B G D E G C B D F E C A

Thank you!

